Assignment 2: Model Predictive Control

Due Mar. 20 at 23:59

This assignment is to be done individually.

Important Note: The university policy on academic dishonesty (cheating) will be taken very seriously in this course. You may not provide or use any solution, in whole or in part, to or by another student.

You are encouraged to discuss the concepts involved in the questions with other students. If you are in doubt as to what constitutes acceptable discussion, please ask! Further, please take advantage of office hours offered by the instructor and the TA if you are having difficulties with this assignment.

DO NOT:

- Give/receive code or proofs to/from other students.
- Copy solution directly from Google or ChatGPT.

DO:

- Meet with other students to discuss assignment (it is best not to take any notes during such meetings, and to re-work assignment on your own).
- Use online resources (e.g. Wikipedia) to understand the concepts needed to solve the assignment.
- Indicate in a cover note how AI tools were used, if at all, to help with the assignment.

Submitting Your Assignment

The assignment must be submitted on Canvas. Your submission must contain a zip file with the following files. These files must be in the root of the zip file.

- `report.pdf`: Written report
- `nmpc_algorithm.py`: nonlinear MPC backend
- `mppi_algorithm.py`: MPPI backend
- `mpc_planner.py`: ROS2 node frontend for nonlinear MPC and MPPI

---

1 Dubins Car in a Convex Corridor

Consider robot described by the Dubins car dynamics

$$\dot{p}_x(t) = v(t)\cos(\theta(t)), \tag{1a}$$
$$\dot{p}_y(t) = v(t)\sin(\theta(t)), \tag{1b}$$
$$\dot{\theta}(t) = \omega(t). \tag{1c}$$

The state is given by $x(t) = (p_x(t), p_y(t), \theta(t))$, where (p_x, p_y) is position and θ is heading (radians). The control is given by $u(t) = (v(t), \omega(t))$, where v is the speed and ω is the turn rate.

Within the finite horizon $t \in [0, T]$, the robot navigates in a corridor defined by

$$y_{\text{low}}(p_x(t)) \leq p_y(t) \leq y_{\text{high}}(p_x(t)), \tag{2a}$$
$$x_{\min} \leq p_x(t) \leq x_{\max}, \tag{2b}$$

using bounded controls

$$v_{\min} \leq v(t) \leq v_{\max}, \tag{3a}$$
$$-\omega_{\max} \leq \omega(t) \leq \omega_{\max}. \tag{3b}$$

Starting from some $x(0) = x_0^{\text{given}}$, the robot aims to reach a goal state $g = (g_x, g_y, g_\theta)$. To model this task, first define the "tracking error" over time to be the distance to goal as follows:

$$e^p(t) = \begin{bmatrix} p_x(t) - g_x \\ p_y(t) - g_y \end{bmatrix}, \tag{4a}$$
$$e^\theta(t) = \theta(t) - g_\theta. \tag{4b}$$

Then, let the running cost $c(\cdot)$ and terminal cost $\ell(\cdot)$ be defined as follows:

$$c(x(t), u(t)) = w_p\|e^p(t)\|_2^2 + w_\theta(e^\theta(t))^2 + w_u\|u(t)\|_2^2, \tag{5a}$$
$$\ell(x(T)) = w_{p,T}\|e^p(T)\|_2^2 + w_{\theta,T}(e^\theta(T))^2, \tag{5b}$$

where $w_p, w_\theta, w_u, w_{p,T}, w_{\theta,T} \in \mathbb{R}$ are weights of the cost function.

(a) Write down a continuous-time optimal control problem over $t \in [0, T]$ that

- starts from the given initial condition $x(0) = x_0$,
- satisfies the corridor and control bounds for all $t \in [0, T]$, and

- minimizes the cost

$$\int_0^T c(x(t), u(t))\, dt + \ell(x(T)). \tag{6}$$

You may assume $y_{\text{low}}(\cdot)$ and $y_{\text{high}}(\cdot)$ are known functions.

(b) Let $T = N\Delta t$ for some given N and Δt. Discretize your continuous-time optimal control problem using **forward Euler** integration with *left-endpoint* evaluation, and write down the resulting finite-dimensional nonlinear program in terms of the decision variables $X_{0:N} = \{x_0, \ldots, x_N\}, U_{0:N-1} = \{u_0, \ldots, u_{N-1}\}$. You should have the following:

- the initial condition constraint
- dynamics constraints at all time points $k = 0, \ldots, N-1$
- corridor constraints at all time points
- control bounds enforced at all time points

2 Nonlinear Model Predictive Control (CasADi)

In this question, you will implement a nonlinear model predictive controller (NMPC) for a Dubins car that must remain within a specified 2D corridor while driving toward a given goal state, as described in Question 1.

Instead of the objective in Question 1, we will use a slightly-modified objective commonly used to produce smoother-varying controls. In discretized form, it is given as follows:

$$\min_{X_{0:N}, U_{0:N-1}} \sum_{k=0}^{N-1} \left(w_{\text{pos}} \|e_k^p\|_2^2 + w_\theta (e_k^\theta)^2 + w_u \|u_k\|_2^2 \right) + \ell(x_N) + \sum_{k=1}^{N-1} w_{\Delta u} \|u_k - u_{k-1}\|^2, \tag{7}$$

Your task is to finish the implementation in `nmpc_algorithm.py`. See `readme.md` in `mpc.zip` for detailed instructions.

Notation:

- Many parameters, including control bounds and weights in the objective, are found in the dictionary `params`. For example, $w_{\Delta u}$ corresponds to `params["w_du"]`.
- Δt corresponds to `dt`.
- Corridor parameters are specified in `corridor_params`. For example,
  - x_{min} corresponds to `corridor_params["x_knots"][0]`
  - x_{max} corresponds to `corridor_params["x_knots"][-1]`
- The goal is specified by `goal`.

Hints (common pitfalls)

- Make sure to use the state ordering `[px, py, theta]` and control ordering `[v, omega]`.
- Enforce corridor constraints for **all** $k = 0, \ldots, N$ (terminal included).
- When building dynamics in CasADi, use CasADi trigonometric functions (not NumPy) if you are working with symbolic variables.
- Be careful with array shapes: many solvers return $(3, N+1)$ for states; you must return $(N+1, 3)$.
- The heading error is **not** wrapped to $[-\pi, \pi]$. Large unwound angles may lead to high costs, but for this assignment you may choose to ignore this issue.
- You may (but are not required to) implement smallest-angle difference for the heading error: `e_theta = ca.atan2(ca.sin(theta - g_theta), ca.cos(theta - g_theta))`
- There is no control cost at the terminal step (no u_N).
- For closed-loop simulation (not required for this question), consider shifting the previous solution as a warm-start for faster convergence.
- If convergence issues arise, you may adjust the solver tolerance to 10^{-5} (default is 10^{-6}), but the defaults should work.
- A correct controller should, for example, drive the car smoothly from $x = 0.5$ to $x = 7.5$ while staying comfortably inside the corridor boundaries.

3 Model Predictive Path Integral

In this question, you will implement *Model Predictive Path Integral* (MPPI), a sampling-based MPC method, for the same Dubins-car corridor navigation setting used in the previous nonlinear MPC assignment.

In addition to the terms in Eq. (7), and *instead of* the (hard) constraints described in Question 1, MPPI adds a penalty term for constraint violation. This running cost is implemented in by the function `dubins_running_cost(...)` which adds the state constraint violation penalty term

$$\begin{aligned}\Phi(p_k) = &\max(0, x_{\min} - p_{x,k})^2 + \max(0, p_{x,k} - x_{\max})^2 \\ &+ \max(0, y_{\text{low}}(p_{x,k}) - p_{y,k})^2 + \max(0, p_{y,k} - y_{\text{high}}(p_{x,k}))^2.\end{aligned} \tag{8}$$

Your task is to finish the implementation in `nmpc_algorithm.py`. See `readme.md` in `mpc.zip` for detailed instructions.

Hints and Notes

- `get_action` returns a NumPy array of shape `(2,)` ordered as `[v, omega]`.
- `rollout` returns a NumPy array of shape `(K, H)`.
- *Shapes:* nominal `self.actions` is `(H, 2)`; sampled actions are `(K, H, 2)`.
- Weights depend on total costs.
- Subtract the lowest total cost before exponentiating to avoid underflow.
- After updating, shift left by one step and duplicate the last action.
- Clip perturbed actions (and clip the updated nominal sequence) to enforce control constraints.

Summary of the MPPI algorithm

MPPI is formed by a (parallel) rollout and an importance sampling update. It maintains a nominal action sequence and iterates over it. Following the lecture notes in section "MPPI: Rollouts & Update", here we will elaborate on some of the details.

Let the nominal action sequence be

$$\mathbf{U} = [u_0, u_1, \dots, u_{H-1}] \tag{9}$$

Then, at each time step, MPPI performs the following:

1. Sample perturbed action sequences: For each sample trajectory $i = 1, \dots, M$ and time step $k = 0, \dots, N-1$, sample noise
$$\varepsilon_{i,k} \sim \mathcal{N}(0, \Sigma) \tag{10}$$
and form perturbed actions with bounds:
$$u_k^i = \text{clip}(u_k + \varepsilon_{i,k}, u_{\min}, u_{\max}). \tag{11}$$
2. Roll out using the dynamics model starting from the current state x_0, and compute per-step and total costs.
3. Compute importance weights via a temperature $\lambda > 0$ (a hyperparameter controlling the sharpness of the weighting). Higher λ makes the weights more uniform (more exploration); lower λ makes the selection focus more on low-cost trajectories (more greedy):
$$w_i = \exp\left(-\frac{1}{\lambda}(S_i - \min_j S_j)\right), \qquad \bar{w}_i = \frac{w_i}{\sum_j w_j}. \tag{12}$$

4. Update nominal control sequence via weighted averaging:

$$u_k \leftarrow \sum_{i=1}^{M} \bar{w}_i u_k^i, \qquad k = 0, \ldots, N-1. \tag{13}$$

5. Receding horizon shift Return the first action u_0 as the control to apply, then shift the nominal plan left by one step and duplicate the last action:

$$[u_0, \ldots, u_{N-1}] \leftarrow [u_1, \ldots, u_{N-1}, u_{N-1}]. \tag{14}$$

4 ROS2 integration

In this question, you will integrate the NMPC and MPPI backends in the previous sections in ROS2 node by finishing the implementation in `mpc_planner.py`. See `readme.md` in `mpc.zip` for detailed instructions.